



Harnesses and Multi-Agent Orchestration

From token prediction to coordinated, rollback-safe agent teams

Sangwhan Moon

June 22, 2026

SECTION

Harnesses 101

Control loops make LLMs usable.

What Is an LLM?

NEXT TOKEN

- Predict next token
- Append and repeat
- Long outputs = chained local bets

AUTOREGRESSIVE LOOP

The cat sat on the mat

distribution over the next token

mat: 0.42 rug: 0.21 floor: 0.14

The Transformer Makes It Work

SELF-ATTENTION

- Read across prior tokens
- Weight what matters now
- Stack attention + MLP blocks

THE COST

- Pairwise token comparisons
- Cost grows $O(n^2)$
- Bigger windows: pricier, noisier

Attention Is All You Need (Vaswani et al., NeurIPS 2017) [1]

Tokens, Context, and Sampling

TOKENS

- Models read tokens, not characters.
- Code, prose, logs, and tool output all spend the same budget.

CONTEXT WINDOW

- Prompt, history, tool output, and response share the window.
- More room is not the same as better control.

SAMPLING

- Temperature changes how broadly the model samples.
- Top-k and top-p trim candidate tokens before sampling.

low temperature: narrow

high temperature: exploratory

same prompt, different useful attempts

References: [1]

How Models Are Built

PRE-TRAINING

Broad next-token competence from large-scale data.

MID-TRAINING

Sharper behavior for domains, code, math, and long-context use.

POST-TRAINING

SFT and preference optimization teach instruction-following behavior.

DEPLOYMENT

Product systems wrap the model with policy, tools, and constraints.

Base Model to Agent

WHAT GETS ADDED

- Instruction tuning
- Tool schemas
- System prompt

WHY IT CHANGES BEHAVIOR

- Request != action
- Harness runs the loop
- Side effects stay controlled

base model -> SFT -> tool use -> system prompt -> harness

What the Harness Owns

CONTROL LOOP

- Manages message history.
- Sends tool schemas to the model.
- Executes tool calls and appends results.

SECURITY BOUNDARY

- Ask, approve, block
- Model proposes actions
- Harness owns execution

messages -> LLM -> tool call -> executor -> result

SFT Teaches Tool Use

TRAINING SAMPLE

```
user: "Find failing tests."  
assistant: tool_call(run_tests, {})  
tool: "2 failures in parser_test"  
assistant: "The parser edge case fails..."
```

PATTERN LEARNED

- When to call tools
- How to pass arguments
- How to chain steps
- How to answer from results

Training traces teach the evidence pattern.

SECTION

Limits of Single Agents

One session. One window. Growing liability.

Single-Agent Harnesses Hit a Ceiling

THE BOTTLENECK

- One window
- One trajectory
- One lens
- Early mistakes compound

one agent

->

plan + files + logs + decisions +
patches

References: [7], [13]

One Context Window Is Not a Plan

CAPACITY IS NOT CONTROL

- More tokens != better focus
- Retrieval, reasoning, instructions compete

FAILURE SHAPE

- Distant facts get buried
- Inspection cost explodes
- Debugging = transcript archaeology

Lost in the Middle (Liu et al., TACL 2024) [14]

Long Inputs Degrade Attention in Practice

NEEDLE TESTS

- Position-sensitive failures
- Longer windows still have cliffs

REAL TASKS

- LongBench spans QA, summary, code
- Compression helps, risk remains

LongBench (Bai et al., ACL 2024) [7]; Lost in the Middle (Liu et al., TACL 2024) [14]

Long Agent Chains Accumulate Errors

CHAIN RISK

- New state every step
- Bad assumptions stick
- Early drift steers the run

MULTI-TURN RISK

- Reliability drops across turns
- Often interaction failure, not capability failure

TOOL CALL

reads files, runs command

->

WRITTEN STATE

summary + assumptions

->

NEXT TURN

state becomes context

small drift -> persisted assumption -> wrong branch

ReAct (Yao et al., ICLR 2023) [5]; LLMs Get Lost In Multi-Turn Conversation (Laban et al., ICLR 2026) [13]

Decomposition Is the Escape Hatch

WHAT WORKS

- Split the task
- Match role + context
- Pass forward only what matters

WHY MODELS ARE GOOD AT IT

- Strong at structure
- Good with intermediates
- L2M / ToT hint at the pattern

large task -> A B C -> synthesis

Least-to-Most (Zhou et al., ICLR 2023) [4]; Tree of Thoughts (Yao et al., NeurIPS 2023) [6]. AgentBench and WebArena evaluate agent task environments [8], [9].

SECTION

Multi-Agent Ensembles

Agent teams adapt ensemble ideas to engineering work: diversity, role fit, critique, and structured synthesis.

Multi-Agents and Ensemble Inspiration

CORE CLAIM

- Test-time ensemble
- Roles + tools + memory boundaries
- Diversity first, synthesis later

fan out -> independent work -> critique -> synthesis

Self-Consistency (Wang et al., ICLR 2023) [3]; More Agents Is All You Need (Li et al., TMLR 2024) [10]

Classical Ensemble Ideas Map Cleanly

BAGGING

Parallel samples, then aggregate.

A -> B -> C -> vote

BOOSTING

Stage work, then correct errors.

draft -> review -> fix

MIXTURE OF EXPERTS

Route by specialty, then fuse outputs.

route -> expert -> fuse

References: [3], [10], [11], [12]

Voting Beats One Guess When Errors Differ

SAMPLE DIVERSITY

- Self-Consistency samples multiple reasoning paths.
- It selects the answer that is most consistent across paths.
- Gains depend on task difficulty and error diversity.

AGENT FOREST

- More Agents Is All You Need scales the same intuition to instantiated agents.
- Parallel attempts are cheap when orchestration is automated.

path A path B path C -> consistent answer

Self-Consistency (Wang et al., ICLR 2023) [3]; More Agents Is All You Need (Li et al., TMLR 2024) [10]

Specialization Beats One Generalist

ROLE FIT

- Different roles, different strengths
- Keep diversity to the merge
- Assign by failure mode

FUSION

- Rank candidates
- Fuse best fragments
- Late merge beats early collapse

architect qa security performance -> synthesizer

LLM-Blender (Jiang et al., ACL 2023) [11]; Mixture-of-Agents (Wang et al., ICLR 2025) [12]

Debate Adds Friction

DELIBERATION

- Chain-of-thought exposes intermediate steps.
- Tree of Thoughts searches paths before committing.

GROUNDING

- ReAct interleaves reasoning and action.
- Collaboration externalizes deliberation across workers.

Chain-of-Thought (Wei et al., NeurIPS 2022) [2]; ReAct (Yao et al., ICLR 2023) [5]; Tree of Thoughts (Yao et al., NeurIPS 2023) [6]

Small Contexts Can Beat One Long Context

WHY IT WORKS

- Smaller prompts, fewer distractions
- Narrower checklists
- Disagreements surface early

WHERE IT FAILS

- Weak synthesis = more text
- Orchestration decides reliability

References: [7], [10], [11], [12]

SECTION

finb Design Decisions

Cheap failure. Visible work. Explicit synthesis.

Design Around Failure

LOCKDOWN

Safe, but the human becomes the bottleneck.

YOLO

Fast, but one bad command can destroy the workspace.

REVERSIBLE

Agents get freedom inside a workspace that can roll back instantly.

Target: controlled blast radius, not constant approval.

Safe-to-Destruct Workspaces

ZFS SNAPSHOTS

- Copy-on-write workspace
- Near-instant rollback
- Reversible destruction

CHROOT

- Filesystem boundary
- No VM required
- Commit or rollback stays explicit

workspace -> snapshot clone -> chroot -> agent run -> commit / rollback

Agent Teams Are a DAG

PHASES

- Plan -> phase -> synthesize
- Dependencies stay explicit

PARALLELISM

- Concurrent tasks
- Bounded context
- Reuse good work

plan -> phase 1 agents -> synthesis -> phase 2 agents -> final

Mailbox Turns Agents Into a Team

MESSAGES

- Agents send mail to named teammates.
- Coordination stays auditable.
- No shared hidden state is required.

DELIVERY

- Mail arrives next phase
- Visible in prompt context
- Influence stays auditable

lead ml data qa -> MAILBOX -> next phase

Synthesis Has Influence Tiers

PRIMARY

Drives the narrative and owns final coherence.

SUPPORTING

Shapes major sections, constraints, and tradeoffs.

CONTRIBUTING

Adds evidence, edge cases, and checks.

tiered outputs

->

attributed or unified
synthesis

->

one deliverable

References: [11], [12]

Iteration and Harness Abstraction

FOLLOW-UP LOOP

- Deliverable -> replan -> rerun
- Fresh agents, fresh context
- Retry only failed work

FOUR LAYERS

- TUI
- Orchestrator
- Agent manager
- Harnesses

SECTION

Example Results

Concrete multi-file builds, not longer chat.

Example Projects

TERMNES

`termnes`

CPU | PPU | mapper | frame

NES emulator: CPU, PPU, mappers

AT3RS

`at3rs`

bitstream -> bands -> PCM

ATRAC3 codec: signal processing

SLOPPNG

`sloppng`

IHDR | IDAT | CRC | CLI

PNG parser, validator, CLI

Multi-file builds with tests and integration points.

What the Results Demonstrate

DECOMPOSITION

Plan -> phases ->
executable work

SPECIALIZATION

Experts own formats,
rendering, QA

VERIFICATION

Review passes catch
integration bugs

parallel work

->

synthesis

->

coherent codebase

->

verification

References

REFERENCES

- [1] Ashish Vaswani et al., Attention Is All You Need. NeurIPS 2017.
- [2] Jason Wei et al., Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. NeurIPS 2022.
- [3] Xuezhi Wang et al., Self-Consistency Improves Chain of Thought Reasoning in Language Models. ICLR 2023.
- [4] Denny Zhou et al., Least-to-Most Prompting Enables Complex Reasoning in Large Language Models. ICLR 2023.
- [5] Shunyu Yao et al., ReAct: Synergizing Reasoning and Acting in Language Models. ICLR 2023.
- [6] Shunyu Yao et al., Tree of Thoughts: Deliberate Problem Solving with Large Language Models. NeurIPS 2023.

References

REFERENCES

- [7] Yushi Bai et al., LongBench: A Bilingual, Multitask Benchmark for Long Context Understanding. ACL 2024.
- [8] Xiao Liu et al., AgentBench: Evaluating LLMs as Agents. ICLR 2024.
- [9] Shuyan Zhou et al., WebArena: A Realistic Web Environment for Building Autonomous Agents. ICLR 2024.
- [10] Junyou Li et al., More Agents Is All You Need. TMLR 2024.
- [11] Dongfu Jiang et al., LLM-Blender: Ensembling Large Language Models with Pairwise Ranking and Generative Fusion. ACL 2023.
- [12] Junlin Wang et al., Mixture-of-Agents Enhances Large Language Model Capabilities. ICLR 2025.
- [13] Philippe Laban et al., LLMs Get Lost In Multi-Turn Conversation. ICLR 2026.
- [14] Nelson F. Liu et al., Lost in the Middle: How Language Models Use Long Contexts. TACL 2024.

Questions?

Harnesses & Multi-Agent Orchestration